

Graph Foundations for DAG-Based Systems

Table of Contents

INTRODUCTION TO GRAPHS	4
1.1 INTRODUCTION: THINKING IN RELATIONSHIPS.....	4
1.2 FROM LISTS TO RELATIONSHIPS.....	4
1.3 WHAT IS A GRAPH?	5
1.4 DIRECTED GRAPHS.....	5
1.5 VISUAL REPRESENTATION	6
1.6 DIRECTED VS UNDIRECTED GRAPHS.....	6
1.7 MERGING PROCESSES.....	7
1.8 IMPLEMENTATION: CREATING, VISUALIZING, AND EXECUTING A GRAPH	7
1.8.1 Representing the Graph in Code	7
1.8.2 Visualizing the Graph.....	8
1.8.3 Executing the Graph (Traversal).....	9
1.8.4 Key Understanding	10
1.8.5 Conclusion of Implementation.....	11
1.9 GRAPHS AS MODELS OF FLOW	11
1.10 SUMMARY	11
TREES: THE SIMPLEST STRUCTURED GRAPH	12
2.1 WHAT IS A TREE?	12
2.2 INTUITION: HIERARCHY IN REAL LIFE	12
2.3 CONVERTING OUR EXAMPLE INTO A TREE	13
2.4 UNDERSTANDING THE STRUCTURE.....	13
2.5 KEY PROPERTIES OF TREES	14
2.6 LEVELS IN A TREE	14
2.7 WHY TREES ARE IMPORTANT	14
2.8 LIMITATION OF TREES	15
2.9 TRANSITION FROM TREES TO GRAPHS	15
2.10 IMPLEMENTATION SECTION: TREE TRAVERSAL	16
2.10.1 Tree Representation	16
2.10.2 Depth-First Traversal of Tree.....	16
2.10.3 Interpretation.....	16
2.11 SUMMARY	17
GRAPH TRAVERSAL — UNDERSTANDING EXECUTION FLOW	18
3.1 WHAT IS GRAPH TRAVERSAL?	18
3.2 INTUITION BEHIND TRAVERSAL.....	18
3.3 DEPTH-FIRST SEARCH (DFS)	19
3.4 BREADTH-FIRST SEARCH (BFS)	20
3.5 KEY DIFFERENCE BETWEEN DFS AND BFS	21
3.6 WHY TRAVERSAL MATTERS	21

3.7 IMPLEMENTATION SECTION: DFS AND BFS	21
3.7.1 Graph Representation	22
3.7.2 Depth-First Search (DFS)	22
3.7.3 Breadth-First Search (BFS)	23
3.8 IMPORTANT INSIGHT	23
3.9 SUMMARY	23
CYCLES IN GRAPHS	25
4.1 WHAT IS A CYCLE?	25
4.2 UNDERSTANDING CYCLES INTUITIVELY	25
4.3 WHY CYCLES ARE DANGEROUS	26
4.4 VISUAL REPRESENTATION OF A CYCLE	26
4.5 CYCLES AND TRAVERSAL	27
4.6 PREVENTING INFINITE TRAVERSAL	27
4.7 CYCLES IN TREES AND GENERAL GRAPHS	28
4.8 CYCLES IN REAL-WORLD SYSTEMS	28
4.9 RECOGNIZING CYCLES	28
4.10 IMPLEMENTATION SECTION: OBSERVING CYCLE BEHAVIOUR	28
4.11 INTERPRETATION	30
4.12 KEY INSIGHT	30
4.13 SUMMARY	30
CYCLE DETECTION	31
5.1 WHAT IS CYCLE DETECTION?	31
5.2 WHY CYCLE DETECTION IS NECESSARY	31
5.3 INTUITION BEHIND DETECTION	32
5.4 THE KEY IDEA: TRACKING THE CURRENT PATH	32
5.5 VISUAL UNDERSTANDING	32
5.6 DFS-BASED CYCLE DETECTION	33
5.7 EXAMPLE USAGE	34
5.7.1 Example With Cycle	34
5.7.2 Example Without Cycle	34
5.8 ALTERNATIVE APPROACH: USING BFS (KAHN'S IDEA)	34
5.9 KEY INSIGHT	35
5.10 SUMMARY	36
TOPOLOGICAL SORTING AND DEPENDENCY GRAPHS	37
6.1 UNDERSTANDING THE NEED FOR ORDERING	37
6.2 WHAT IS TOPOLOGICAL SORTING?	37
6.3 PROPERTIES OF TOPOLOGICAL ORDERING	38
6.4 EXAMPLE OF TOPOLOGICAL ORDERING	38
6.5 WHY CYCLES PREVENT TOPOLOGICAL SORTING	39
6.6 METHOD 1: KAHN'S ALGORITHM (IN-DEGREE METHOD)	39
6.6.1 Core Idea	39
6.6.2 Intuition	39
6.6.3 Step-by-Step Illustration	40
6.7 IMPLEMENTATION OF KAHN'S ALGORITHM	41
6.8 EXAMPLE EXECUTION	42
6.9 METHOD 2: DFS-BASED TOPOLOGICAL SORTING	42

6.9.1 Core Idea	42
6.9.2 Intuition.....	42
6.9.3 Implementation	43
6.10 DEPENDENCY GRAPHS	43
6.11 KEY INSIGHT	44
6.12 SUMMARY	44
DIRECTED ACYCLIC GRAPHS (DAGS) AND STATE FLOW	45
7.1 WHAT IS A DAG?	45
7.2 WHY DAGs ARE IMPORTANT.....	45
7.3 INTUITION USING OUR RUNNING EXAMPLE	46
7.4 VISUAL REPRESENTATION OF A DAG	46
7.5 DAG VS GENERAL GRAPH	47
7.6 DAG AND TOPOLOGICAL ORDERING	47
7.7 UNDERSTANDING STATE FLOW IN GRAPHS.....	47
7.8 WHAT IS STATE FLOW?	48
7.9 KEY CHARACTERISTICS OF STATE FLOW IN DAGs	49
7.10 DAGs IN REAL-WORLD SYSTEMS	50
7.11 IMPLEMENTATION SECTION: VERIFYING A DAG AND EXECUTING FLOW	50
7.12 KEY INSIGHT	52
7.13 SUMMARY	52
LANGGRAPH AND AGENT PLANNING: APPLYING GRAPH FOUNDATIONS	53
8.1 WHAT IS LANGGRAPH? (CONCEPTUAL VIEW).....	53
8.2 MAPPING GRAPH CONCEPTS TO AI SYSTEMS.....	53
8.3 EXAMPLE: AI CHATBOT PIPELINE	54
8.4 INTRODUCING BRANCHING IN AGENT SYSTEMS.....	55
8.5 WHY DAGs ARE CRITICAL IN LANGGRAPH	56
8.6 STATE FLOW IN AGENT SYSTEMS.....	57
8.7 IMPLEMENTATION SECTION: SIMPLE AGENT WORKFLOW	58
8.8 MINI PROJECT	59
8.9 FINAL SUMMARY	60

Introduction to Graphs

1.1 Introduction: Thinking in Relationships

Before studying graphs formally, it is important to understand why they exist. Many learners try to memorize definitions, but graphs are fundamentally about **relationships between things**.

Consider preparing a sandwich and a cup of tea. The steps involved may include:

- Get bread
- Add butter
- Boil water
- Add tea leaves

At first glance, this looks like a simple list. However, these steps are not independent. They are connected by dependencies.

You cannot add butter before getting bread. You cannot add tea leaves before boiling water.

This leads to a key insight: Real-world processes are not just lists, they are **networks of dependent actions**. To represent such networks clearly, we use **graphs**.

1.2 From Lists to Relationships

Instead of writing tasks as a list, we represent dependencies:

Get Bread → Add Butter → Sandwich Ready

Boil Water → Add Tea Leaves → Tea Ready

This representation shows:

- Order of execution
- Independent processes
- Dependencies clearly

A list forces everything into one sequence. A graph allows **parallel structure and dependency modeling**.

1.3 What is a Graph?

A graph consists of:

- **Nodes (Vertices):** Entities (tasks, states, objects)
- **Edges:** Relationships between nodes

In this example:

- Nodes = tasks
- Edges = “must happen before”

1.4 Directed Graphs

When relationships have direction, we use **directed graphs**.

Example:

Boil Water → Add Tea Leaves

This means:

- First: Boil Water
- Then: Add Tea Leaves

Directed graphs are essential in:

- Task execution
- Workflows
- AI pipelines

1.5 Visual Representation

Figure 1.5.1 shows Directed Graph for Cooking Workflow

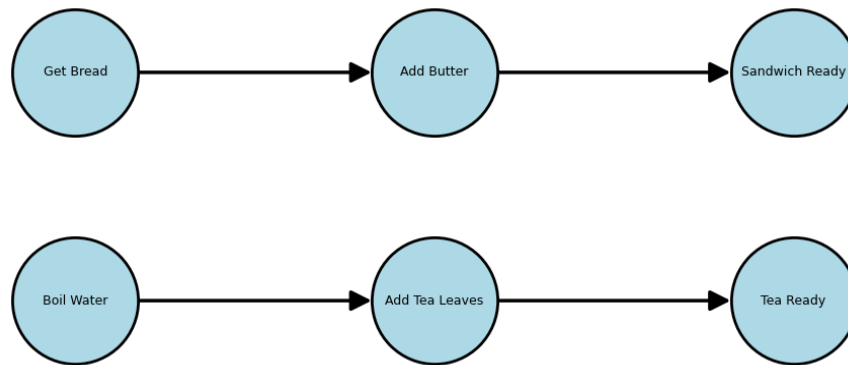


Figure 1.5.1 Directed graph representing task dependencies in cooking workflow.

Two independent chains are shown:

- Sandwich preparation
- Tea preparation

Arrows indicate order.

1.6 Directed vs Undirected Graphs

Not all relationships are directional.

- Directed: $A \rightarrow B$ (order matters)
- Undirected: $A - B$ (mutual relationship)

Figure 1.6.1 shows Directed vs Undirected Graph



*Figure 1.6.1 Comparison of directed and undirected graphs.
Left: Directed graph with arrow; Right: Undirected graph without arrow*

1.7 Merging Processes

We now combine both workflows:

Sandwich → Serve Meal

Tea → Serve Meal

Combined Workflow Graph can be seen in Figure 1.7.1. This introduces **multiple paths merging into one node**.

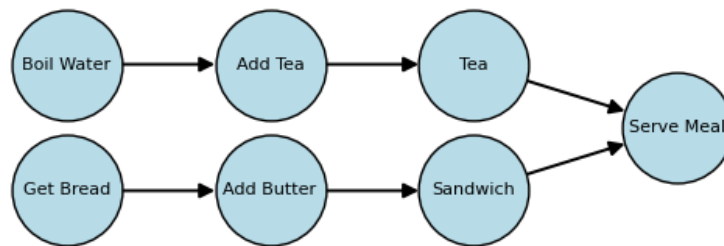


Figure 1.7.1 Graph showing merging dependencies.

Two branches merge into "Serve Meal"

1.8 Implementation: Creating, Visualizing, and Executing a Graph

So far, we have understood graphs conceptually and visually. In this section, we now translate that understanding into a working implementation.

A graph in a program has two aspects:

1. **Internal Representation:** how the graph is stored in code
2. **External Behavior:** how the graph is visualized and executed

1.8.1 Representing the Graph in Code

We begin by storing the graph as a dictionary. Each key represents a node, and its value represents the nodes it points to.

Python Code:

```
graph = {
    "Get Bread": ["Add Butter"],
    "Add Butter": ["Sandwich"],
    "Boil Water": ["Add Tea"],
    "Add Tea": ["Tea"],
    "Sandwich": ["Serve Meal"],
    "Tea": ["Serve Meal"],
    "Serve Meal": []
}
```

This structure captures all dependencies:

- Each node knows what comes next
- The relationships are stored explicitly

However, this representation alone does not produce any visible output. It simply defines the graph internally.

1.8.2 Visualizing the Graph

To actually see the graph, we use a visualization library.

Python Code:

```
import networkx as nx
import matplotlib.pyplot as plt

G = nx.DiGraph()
edges = [
    ("Get Bread", "Add Butter"),
    ("Add Butter", "Sandwich"),
    ("Boil Water", "Add Tea"),
    ("Add Tea", "Tea"),
    ("Sandwich", "Serve Meal"),
    ("Tea", "Serve Meal")
]

G.add_edges_from(edges)
pos = nx.spring_layout(G, seed=42)

nx.draw(
    G, pos,
    with_labels=True, node_color="skyblue",
    node_size=3000, font_size=9, arrows=True)

plt.title("Graph Visualization from Code")
plt.show()
```

Output Explanation

When this code is executed, a window appears displaying a graph with:

- Nodes labeled with task names
- Arrows showing dependencies
- A structure matching the workflow described earlier

This is the **visual representation of the graph**.

Figure 1.8.1 shows Graph Visualization Generated from Code

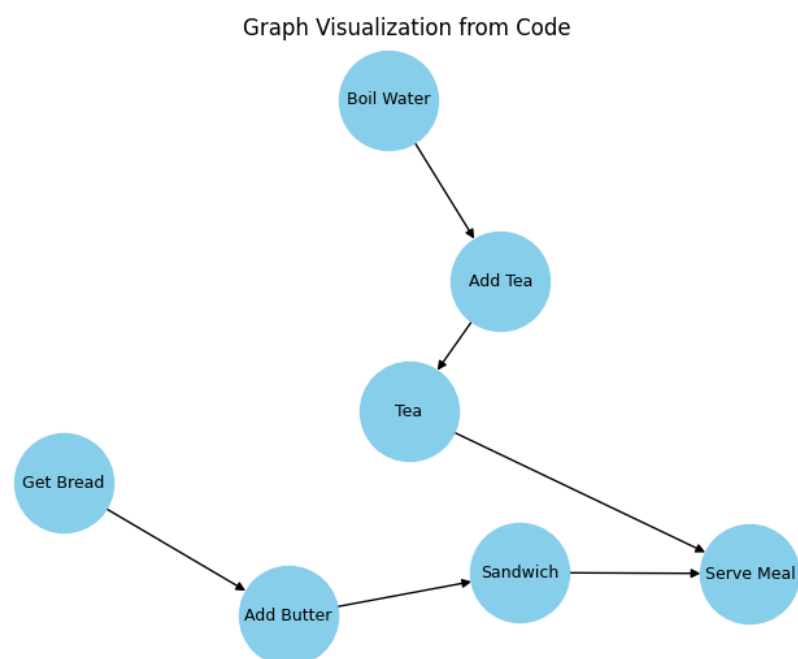


Figure 1.8.1: A programmatically generated directed graph representing task dependencies.

This figure shows the complete cooking workflow graph. Nodes represent tasks such as “Get Bread”, “Add Butter”, and “Serve Meal”. Directed arrows indicate the flow of execution from one task to another.

1.8.3 Executing the Graph (Traversal)

Once the graph is created, we can simulate how tasks are executed. One simple way to do this is using Depth-First Search (DFS).

Python Code for Traversal:

```
def dfs(graph, node, visited):  
    if node not in visited:  
        print(node)  
        visited.add(node)  
  
        for neighbor in graph[node]:  
            dfs(graph, neighbor, visited)  
  
visited = set()  
dfs(graph, "Get Bread", visited)
```

Output of Traversal

When this code is executed, the output will be:

```
Get Bread  
Add Butter  
Sandwich  
Serve Meal
```

Interpretation of Output

This output represents the **execution flow** of one branch of the graph. It shows the order in which tasks are processed when starting from “Get Bread”.

1.8.4 Key Understanding

It is important to distinguish between different aspects of a graph:

- The **dictionary structure** stores the graph
- The **visualization code** produces an image
- The **traversal code** produces an execution sequence

Each serves a different purpose:

Component	Purpose
Graph structure	Stores relationships
Visualization	Shows the graph
Traversal	Executes the graph

1.8.5 Conclusion of Implementation

Through this section, we move from theory to practice. We see that a graph is not just an abstract idea, but a structure that can be:

- Stored
- Visualized
- Executed

This ability to represent and execute graphs is what makes them powerful in real-world systems, including workflow engines and AI pipelines.

1.9 Graphs as Models of Flow

A graph represents:

- Flow of tasks
- Flow of data
- Flow of execution

Each edge tells us: “Where to go next”

1.10 Summary

In this chapter, we learnt:

- Why graphs exist
- How dependencies define structure using nodes and edges
- Directed vs undirected graphs
- Graph as execution flow
- Basic graph implementation

Trees: The Simplest Structured Graph

In Chapter 1, we introduced graphs as structures that represent relationships and dependencies between tasks. We saw how real-world processes, such as preparing a meal, can be modelled using nodes and directed edges.

However, the graphs we studied had a certain level of flexibility:

- Multiple starting points
- Independent branches
- Merging paths

While this flexibility is powerful, it can also make graphs difficult to understand for beginners. To build stronger intuition, we now study a simpler and more structured form of graphs: a **tree**

2.1 What is a Tree?

A **tree** is a special type of graph with the following properties:

1. It has a **single starting point**, called the root
2. It has **no cycles** (no loops)
3. Each node has **exactly one parent** (except the root)
4. There is exactly **one path between any two nodes**

In simple terms, a tree is a graph that looks like a hierarchy.

2.2 Intuition: Hierarchy in Real Life

Trees appear naturally in many real-world systems:

- Family trees
- Organization charts
- File systems in a computer
- Decision-making processes

All of these follow a **top-down structure**, starting from one root and branching outward.

2.3 Converting Our Example into a Tree

Let us revisit our cooking example. Earlier, we had two independent processes:

- Sandwich preparation
- Tea preparation

Now, we introduce a single root node: **Make Meal**

From this root, the process branches into two parts.

Figure 2.3.1 shows a tree structure where all tasks originate from a single root node and branch into sub-tasks.

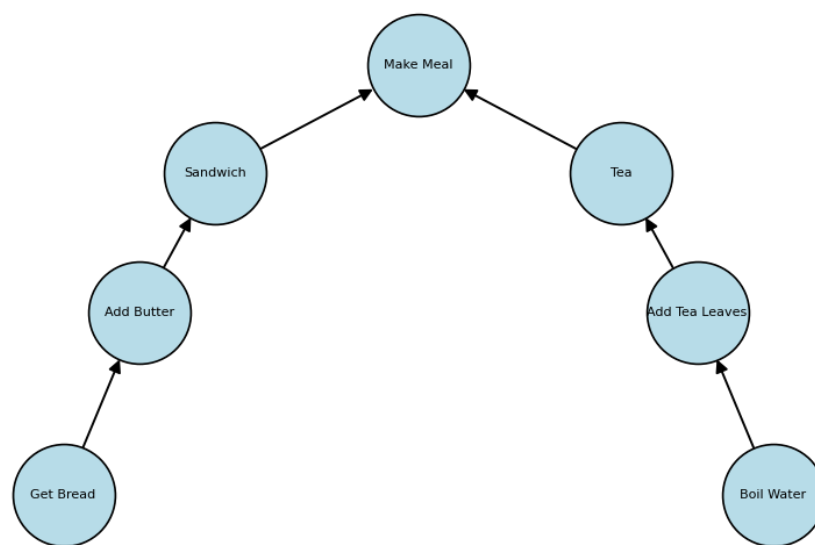


Figure 2.3.1: Tree Representation of Meal Preparation

The diagram begins with a root node labeled “Make Meal”. From this node, two branches extend horizontally to “Sandwich” and “Tea”. Further branches extend downward: “Sandwich” connects to “Get Bread” and “Add Butter”, while “Tea” connects to “Boil Water” and “Add Tea Leaves”.

2.4 Understanding the Structure

In this tree:

- “Make Meal” is the **root node**
- “Sandwich” and “Tea” are **child nodes**
- “Get Bread” and “Boil Water” are **leaf nodes**

A **leaf node** is a node that has no children.

2.5 Key Properties of Trees

Let us examine the defining characteristics more closely.

1. Single Root

Every tree starts from one node.

2. No Cycles

You cannot return to a node once you leave it.

3. Unique Path

There is only one way to reach any node.

4. Hierarchical Structure

Nodes are arranged in levels.

2.6 Levels in a Tree

Trees are often understood in terms of levels:

- Level 0: Root node
- Level 1: Children of root
- Level 2: Grandchildren

In our example:

- Level 0 → Make Meal
- Level 1 → Sandwich, Tea
- Level 2 → Add Butter, Add Tea Leaves
- Level 3 → Get Bread, Boil Water

2.7 Why Trees Are Important

Trees are easier to understand because:

- They eliminate ambiguity
- They provide a clear structure
- They simplify traversal

For beginners, trees serve as a stepping stone to understanding more complex graphs.

2.8 Limitation of Trees

While trees are simple, they are also limited.

Consider this situation:

Both **Tea** and **Coffee** require boiling water.

In a tree:

- “Boil Water” would need two parents
- But trees do not allow multiple parents

This breaks the structure.

Figure 2.8.1 illustrates a situation where a tree structure fails due to shared dependencies.

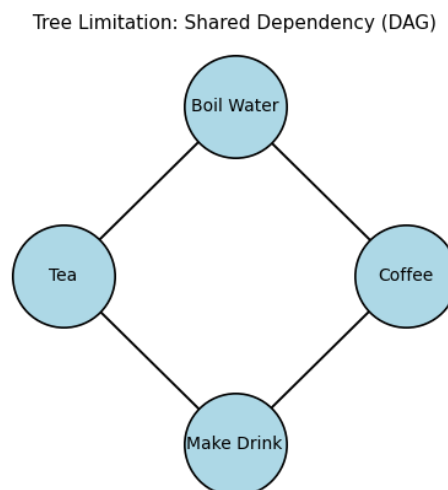


Figure 2.8.1 Limitation of Tree Structure

The diagram attempts to connect “Boil Water” to both “Tea” and “Coffee”. This creates a structure where one node has multiple parents, violating tree rules.

2.9 Transition from Trees to Graphs

This limitation leads to an important realization that trees are not sufficient to represent all real-world systems.

When a node has multiple dependencies, processes merge, and relationships are more complex, we must move beyond trees to **general graphs**.

2.10 Implementation Section: Tree Traversal

To understand trees computationally, we now implement a simple traversal.

2.10.1 Tree Representation

```
tree = {  
    "Make Meal": ["Sandwich", "Tea"],  
    "Sandwich": ["Get Bread", "Add Butter"],  
    "Tea": ["Boil Water", "Add Tea Leaves"],  
    "Get Bread": [],  
    "Add Butter": [],  
    "Boil Water": [],  
    "Add Tea Leaves": []  
}
```

2.10.2 Depth-First Traversal of Tree

```
def traverse_tree(tree, node):  
    print(node)  
  
    for child in tree[node]:  
        traverse_tree(tree, child)  
  
traverse_tree(tree, "Make Meal")
```

Output

```
Make Meal  
Sandwich  
Get Bread  
Add Butter  
Tea  
Boil Water  
Add Tea Leaves
```

2.10.3 Interpretation

This output shows how the tree is explored from root to leaves.

It reflects:

- Hierarchical execution
- Structured flow
- No ambiguity

2.11 Summary

In this chapter, we introduced trees as a special type of graph.

We learned that:

- Trees are hierarchical graphs
- They have a single root and no cycles
- They provide a clear and simple structure
- They are useful for modeling structured processes
- However, they cannot represent shared dependencies

Graph Traversal — Understanding Execution Flow

In Chapter 1 and 2, we learned how graphs represent relationships and dependencies and simplified graphs into trees to understand hierarchical structure.

Now, a natural question arises:

Once a graph is built, **how do we actually use it?**

In real systems, we do not just store graphs — we **execute them**.

Execution means:

- Visiting nodes
- Following edges
- Processing tasks in a meaningful order

This process is called **graph traversal**.

3.1 What is Graph Traversal?

Graph traversal is the process of systematically visiting nodes in a graph.

It answers questions like:

- In what order should tasks be executed?
- How do we explore all parts of a graph?
- How does control flow through a system?

There are two fundamental strategies:

1. **Depth-First Search (DFS)**
2. **Breadth-First Search (BFS)**

3.2 Intuition Behind Traversal

Let us return to our running example.

We now simplify it into a graph:

Start → A → B
↓
C

The key question is:

Should we go deep first or wide first?

This leads to two different behaviours.

3.3 Depth-First Search (DFS)

Depth-First Search goes as deep as possible before coming back.

Step-by-Step Execution

Starting from “Start”:

1. Visit Start
2. Go to A
3. Go to B (deepest path)
4. Backtrack to A
5. Visit C

DFS Order

Start → A → B → C

Figure 3.3.1 illustrates Depth-First Search, where the traversal explores one branch completely before moving to another.

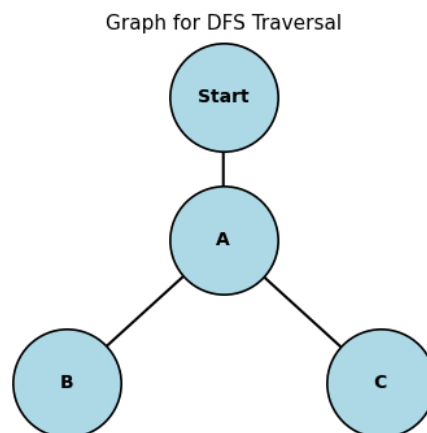


Figure 3.3.1: DFS Traversal

The graph is arranged horizontally. Starting from “Start”, the traversal moves to “A”, then continues to “B” before returning and visiting “C”. The path shows deep exploration before moving sideways.

3.4 Breadth-First Search (BFS)

Core Idea

Breadth-First Search follows this rule:

Explore all neighbours before going deeper.

Step-by-Step Execution

Starting from “Start”:

1. Visit Start
2. Visit A
3. Visit B and C (same level)

BFS Order

Start → A → B → C

Figure 3.4.1 illustrates Breadth-First Search, where nodes are explored level by level.

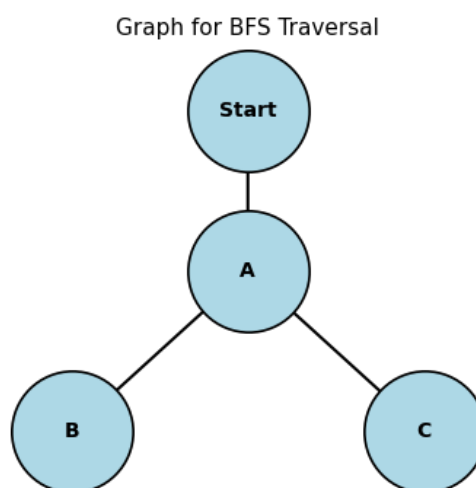


Figure 3.4.1 Breadth-First Traversal

The graph is the same as before, but traversal proceeds level by level. After visiting “Start”, the algorithm visits “A”, and then explores both “B” and “C” before going deeper.

An other example is given in Section 3.7 for better understanding.

3.5 Key Difference Between DFS and BFS

Aspect	DFS	BFS
Strategy	Go deep first	Go level by level
Memory	Uses recursion/stack	Uses queue
Behavior	Explores paths fully	Explores layers

3.6 Why Traversal Matters

Traversal is not just a technical operation. It represents:

- **Execution order in programs**
- **Task scheduling in workflows**
- **Decision-making in AI systems**

For example:

- DFS resembles **deep reasoning or backtracking**
- BFS resembles **parallel exploration**

3.7 Implementation Section: DFS and BFS

We now implement both traversal methods.

Consider the graph given in Figure 3.7.1 for which we will perform the traversal methods.

Example Graph for BFS and DFS Traversal

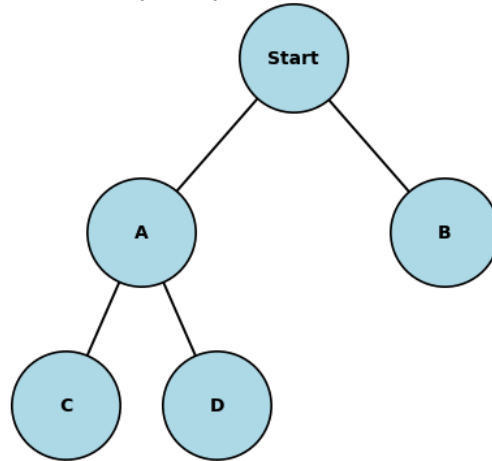


Figure 3.7.1: Example Graph for Traversals

3.7.1 Graph Representation

```
tree = {  
    "Start": ["A", "B"],  
    "A": ["C", "D"],  
    "B": [],  
    "C": [],  
    "D": []  
}
```

3.7.2 Depth-First Search (DFS)

```
def dfs(tree, node, visited):  
    if node not in visited:  
        print(node)  
        visited.add(node)  
        for child in tree[node]:  
            dfs(tree, child, visited)  
visited = set()  
dfs(tree, "Start", visited)
```

Output (DFS)

Start
A
C
D
B

3.7.3 Breadth-First Search (BFS)

```
from collections import deque
def bfs(tree, start):
    visited = set()
    queue = deque([start])

    while queue:
        node = queue.popleft()
        if node not in visited:
            print(node)
            visited.add(node)
            queue.extend(tree[node])

bfs(tree, "Start")
```

Output (BFS)

Start

A

B

C

D

3.8 Important Insight

Even when outputs look similar, the **process is different**:

- DFS explores one path fully
- BFS explores all possibilities evenly

This difference becomes critical in:

- Large graphs
- Decision systems
- AI planning

3.9 Summary

In this chapter, we learned:

- Graph traversal defines execution

- DFS explores deeply
- BFS explores broadly
- Traversal determines how systems behave
- Graphs are not just stored — they are executed

In the next chapter, we encounter a critical issue: Cycles — when graphs loop back on themselves.

Cycles in Graphs

In the previous chapters, we gradually built our understanding of graph-based structures. In Chapter 3, we studied traversal methods such as Depth-First Search (DFS) and Breadth-First Search (BFS), which define how we move through a graph.

All the discussions implicitly assumed something important:

The graph can be traversed in a way that eventually **finishes**.

However, this assumption does not always hold true.

There exists a structure within graphs that can completely break execution: A **cycle**

4.1 What is a Cycle?

A **cycle** is a path in a graph that starts and ends at the same node.

In other words, if you can begin at a node, follow a sequence of edges, and return to the same node, a cycle exists.

Example

$A \rightarrow B \rightarrow C \rightarrow A$

This forms a loop:

- A leads to B
- B leads to C
- C leads back to A

4.2 Understanding Cycles Intuitively

To understand cycles clearly, let us revisit our cooking example and introduce a logical mistake.

Suppose we define the process as:

- Make Tea depends on Boil Water
- Boil Water depends on Add Tea Leaves
- Add Tea Leaves depends on Make Tea

This creates the following loop:

Make Tea → Boil Water → Add Tea Leaves → Make Tea

Now consider what happens:

- To make tea, you need boiling water
- To boil water, you need tea leaves added
- To add tea leaves, you must already be making tea

This creates a circular dependency with no starting point.

4.3 Why Cycles Are Dangerous

Cycles introduce several critical problems in systems.

1. Infinite Execution

When traversal reaches a cycle, it can continue indefinitely:

A → B → C → A → B → C → ...

The process never terminates.

2. Logical Inconsistency

A task may depend on itself indirectly. This makes execution logically impossible.

3. System Failures

In programming systems, cycles can lead to:

- Infinite recursion
- Memory overflow
- Program crashes

4.4 Visual Representation of a Cycle

Figure 4.4.1 shows a cycle where traversal returns to the starting node, creating a loop.

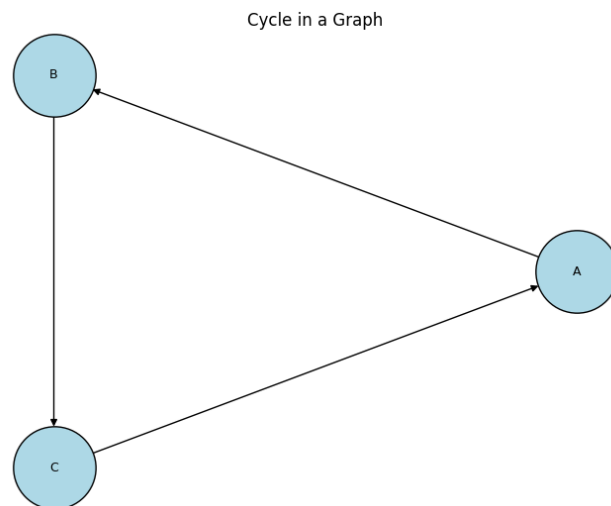


Figure 4.4.1: Simple Cycle in a Graph

The diagram consists of three nodes labelled A, B, and C. Arrows connect A to B, B to C, and C back to A, forming a closed loop.

4.5 Cycles and Traversal

Let us examine how traversal behaves in the presence of a cycle.

Consider using Depth-First Search (DFS):

- Start at A
- Move to B
- Move to C
- From C, return to A

Now the traversal repeats: $A \rightarrow B \rightarrow C \rightarrow A \rightarrow B \rightarrow C \rightarrow \dots$

This results in an infinite loop unless additional control is applied.

4.6 Preventing Infinite Traversal

To prevent such behaviour, traversal algorithms use a mechanism called:

Visited Node Tracking

When a node is visited:

- It is recorded
- It is not processed again

This ensures that traversal eventually stops.

4.7 Cycles in Trees and General Graphs

From Chapter 2, we know that trees are special graphs that do not contain cycles. This leads to an important comparison:

Property	Tree	General Graph
Cycles	Not allowed	Allowed
Effect on traversal	Always terminates naturally	May loop infinitely if cycles are not handled
Structural implication	Strictly hierarchical	Flexible and interconnected

Trees guarantee safe execution, while general graphs require careful handling.

4.8 Cycles in Real-World Systems

Cycles are not just theoretical concepts. They appear in many practical situations:

1. **Software Dependencies**

Module A depends on B, B depends on C, and C depends on A.

2. **Deadlocks in Operating Systems**

Process A waits for B, and B waits for A.

3. **Workflow Systems**

Task execution becomes stuck due to circular dependencies.

4.9 Recognizing Cycles

A cycle exists if:

There is a path from a node back to itself.

In small graphs, cycles are easy to identify visually. However, in large systems, they are not obvious, which makes detection essential.

4.10 Implementation Section: Observing Cycle Behaviour

In this section, we explore how cycles affect execution through code.

Graph with a Cycle

```
graph = {  
    "A": ["B"],  
    "B": ["C"],  
    "C": ["A"]  
}
```

DFS Without Safety Check

```
def dfs_no_check(graph, node):  
    print(node)  
  
    for neighbor in graph[node]:  
        dfs_no_check(graph, neighbor)  
  
dfs_no_check(graph, "A")
```

Expected Behavior

This program will:

- Continue running indefinitely
- Eventually stop with a recursion error

This demonstrates how cycles can break execution.

DFS With Visited Check

```
def dfs_safe(graph, node, visited):  
    if node not in visited:  
        print(node)  
        visited.add(node)  
  
    for neighbor in graph[node]:  
        dfs_safe(graph, neighbor,  
visited)  
  
visited = set()  
dfs_safe(graph, "A", visited)
```

Output

A
B
C

4.11 Interpretation

By introducing a visited set:

- Each node is processed only once
- Infinite loops are prevented

This simple idea is fundamental to safe graph traversal.

4.12 Key Insight

Cycles are not always visible, but their impact is significant.

A graph with cycles requires control mechanisms to ensure safe execution.

Without such mechanisms, systems can fail unpredictably.

4.13 Summary

In this chapter, we explored cycles in graphs.

We learned that:

- A cycle is a loop in a graph
- Cycles can cause infinite execution
- Traversal must handle cycles carefully
- Trees are cycle-free and safe

In the next chapter, we move from understanding the problem to solving it: Cycle Detection

Cycle Detection

In Chapter 4, we introduced the concept of **cycles** and understood why they are dangerous.

We saw that:

- Cycles create infinite loops
- They lead to logical inconsistencies
- They can break traversal algorithms

However, simply knowing that cycles are harmful is not enough.

In real systems, graphs can be large and complex. It is not always possible to visually inspect them and identify cycles.

This leads to an important requirement:

Before executing a graph, we must **detect whether a cycle exists**.

5.1 What is Cycle Detection?

Cycle detection is the process of determining whether a graph contains any cycles.

In simple terms:

We check whether any node can be reached again through a path starting from itself.

If such a path exists, the graph is **not safe for certain operations**, especially those that require a clear execution order.

5.2 Why Cycle Detection is Necessary

Cycle detection is critical in many real-world systems:

1. Task Scheduling

Tasks must be executed in a valid order. Cycles make this impossible.

2. Build Systems

If file A depends on B, and B depends on A, compilation fails.

3. AI Pipelines

Execution flow must be acyclic to avoid infinite loops.

5.3 Intuition Behind Detection

Let us think about how we might detect a cycle while traversing a graph.

When performing DFS:

- We move from one node to another
- We keep track of visited nodes

But simply marking nodes as visited is not enough. Because revisiting a node later does not always mean a cycle. It may simply be reached through a different path.

5.4 The Key Idea: Tracking the Current Path

To detect cycles correctly, we track:

1. Nodes that have been visited before
2. Nodes that are part of the **current traversal path**

A cycle exists if: During traversal, we encounter a node that is already in the current path.

5.5 Visual Understanding

Figure 5.5.1 shows how revisiting a node in the current path indicates a cycle.

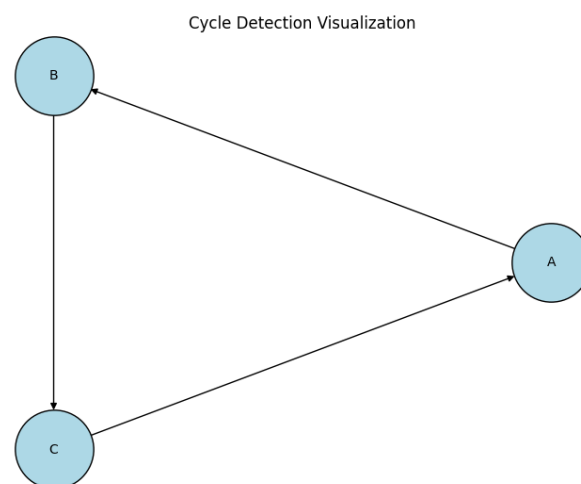


Figure 5.5.1: Cycle Detection Using DFS Path Tracking

The graph contains nodes A, B, and C forming a cycle. During traversal from $A \rightarrow B \rightarrow C$, the algorithm attempts to revisit A, which is already part of the current path.

5.6 DFS-Based Cycle Detection

We now formalize the idea.

During DFS, we maintain:

- A **visited set** (nodes already processed)
- A **recursion stack (path set)**

Python Code:

```
def detect_cycle(graph):
    visited = set()
    path = set() # recursion stack

    def dfs(node):
        if node in path:
            return True # cycle detected
        if node in visited:
            return False

        visited.add(node)
        path.add(node)

        for neighbor in graph[node]:
            if dfs(neighbor):
                return True

        path.remove(node)
        return False

    for node in graph:
        if dfs(node):
            return True

    return False

# Example usage
graph_with_cycle = {
    "A": ["B"],
    "B": ["C"],
    "C": ["A"]
}

print(detect_cycle(graph_with_cycle))
```

Output:

True

5.7 Example Usage

From Python Implementation given in Section 5.6

5.7.1 Example With Cycle

```
graph = {
    "A": ["B"],
    "B": ["C"],
    "C": ["A"]
}

print(detect_cycle(graph))
```

Output

True

5.7.2 Example Without Cycle

```
graph = {
    "A": ["B"],
    "B": ["C"],
    "C": []
}

print(detect_cycle(graph))
```

Output

False

5.8 Alternative Approach: Using BFS (Kahn's Idea)

Another way to detect cycles is based on **in-degree**.

An **incoming edge** is an edge that points **towards a node**.

The **in-degree** of a node is the **number of incoming edges** it has.

- A node with **in-degree 0** does not depend on any other node.
- A node with **in-degree ≥ 1** depends on at least one other node.

Idea:

- Count incoming edges for each node
- Remove nodes with zero in-degree
- Repeat

If nodes remain at the end: A cycle exists

Nodes with zero incoming edges are removed step by step. In a cyclic graph, some nodes remain that cannot be removed.

Figure 5.8.1 shows how remaining nodes after removal indicate a cycle.

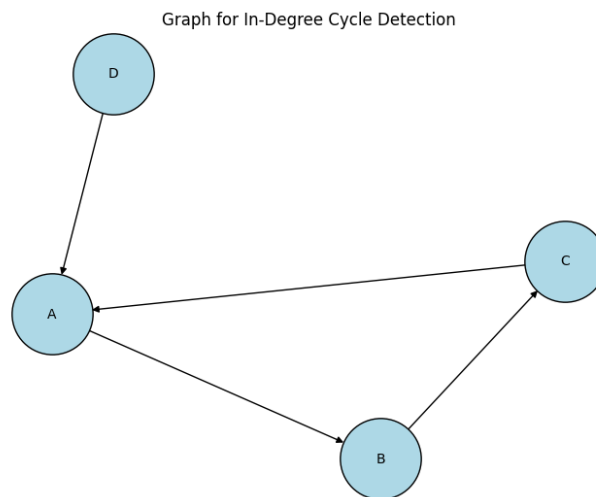


Figure 5.8.1 In-Degree Based Cycle Detection

- Node D has in-degree 0, so it is removed first.
- After removing D, nodes A, B, and C still have incoming edges.
- These remaining nodes form a cycle and cannot be eliminated.

Hence, the remaining nodes indicate the presence of a cycle.

5.9 Key Insight

Cycle detection ensures:

- Safe execution
- Valid dependency structure
- Prevention of infinite loops

Without cycle detection, systems cannot guarantee correctness.

5.10 Summary

In this chapter, we learned:

- Cycles must be detected before execution
- DFS can detect cycles using path tracking
- Re-visiting a node in the current path indicates a cycle
- In-degree methods provide an alternative approach
- Cycle detection is essential in real-world systems

Now that we can detect cycles, we move to the next important concept: Topological Sorting.

Topological Sorting and Dependency Graphs

In the previous chapter, we learned how to detect cycles in a graph and understood why they are problematic. A graph containing cycles cannot be executed in a consistent and finite manner because it creates circular dependencies.

Only graphs **without cycles** can support a clear execution order.

Such graphs allow us to determine how tasks should be performed step by step. Once we ensure that a graph is free of cycles, the next logical step is to find an appropriate sequence in which its nodes can be processed.

This sequence is known as **Topological Ordering**. The process of finding such an order is called **Topological Sorting**.

6.1 Understanding the Need for Ordering

Let us return to our running example of preparing a meal. Consider the following dependencies:

- Get Bread → Add Butter
- Add Butter → Serve Sandwich
- Boil Water → Add Tea Leaves
- Add Tea Leaves → Serve Tea

If we attempt to execute these tasks randomly, we may violate dependencies. For example, serving a sandwich before adding butter is incorrect.

Thus, we require an ordering such that:

- Every task is performed **after its prerequisites**
- No dependency is violated

This requirement naturally leads to the concept of topological sorting.

6.2 What is Topological Sorting?

Topological sorting is a linear arrangement of the nodes of a directed graph such that:

For every directed edge $A \rightarrow B$, node A appears before node B in the ordering.

This ensures that all dependencies are respected.

6.3 Properties of Topological Ordering

Topological ordering has several important characteristics:

1. It exists only for **Directed Acyclic Graphs (DAGs)**
2. There may be **multiple valid orderings**
3. It guarantees **dependency satisfaction**

6.4 Example of Topological Ordering

Consider the following graph:

Get Bread → Add Butter → Sandwich Ready

Boil Water → Add Tea Leaves → Tea Ready

A valid ordering could be:

Get Bread → Boil Water → Add Butter → Add Tea Leaves → Sandwich Ready → Tea Ready

Another valid ordering could be:

Boil Water → Get Bread → Add Tea Leaves → Add Butter → Tea Ready → Sandwich Ready

Both are correct because all dependencies are satisfied.

Figure 6.4.1 illustrates a graph where topological sorting determines a valid sequence of execution.

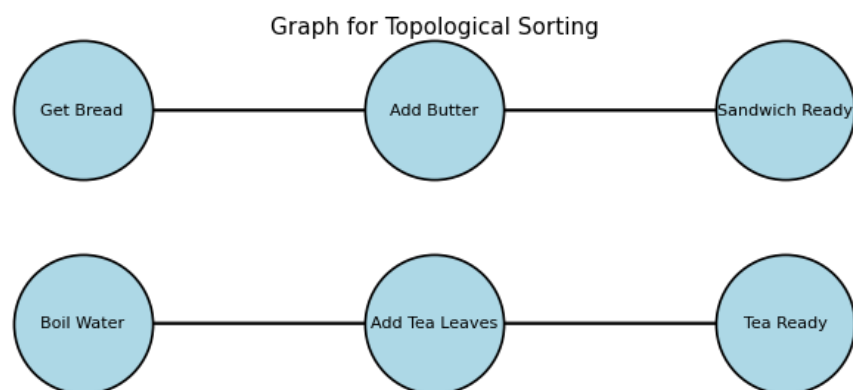


Figure 6.4.1 Graph for Topological Sorting

The diagram shows two independent chains of tasks: one for sandwich preparation and one for tea preparation. Arrows indicate the required order within each chain.

6.5 Why Cycles Prevent Topological Sorting

If a cycle exists, topological ordering becomes impossible.

Consider:

$A \rightarrow B \rightarrow C \rightarrow A$

Here:

- A must come before B
- B must come before C
- C must come before A

This creates a contradiction. No valid ordering can satisfy all conditions.

Thus:

Topological sorting is possible **only when no cycles exist**.

6.6 Method 1: Kahn's Algorithm (In-Degree Method)

One of the most widely used methods for topological sorting is **Kahn's Algorithm**.

6.6.1 Core Idea

The algorithm works as follows:

1. Identify nodes with **no incoming edges (in-degree = 0)**
2. Remove them from the graph
3. Reduce the in-degree of their neighbours
4. Repeat until all nodes are processed

6.6.2 Intuition

Nodes with no incoming edges:

- Do not depend on any other task

- Can be executed immediately

6.6.3 Step-by-Step Illustration

Consider the graph:

$A \rightarrow B \rightarrow D$

$A \rightarrow C \rightarrow D$

Steps:

1. A has in-degree 0 $\rightarrow$ process A
2. B and C now have in-degree 0 $\rightarrow$ process them
3. D becomes 0 $\rightarrow$ process D

Figure 6.6.1 shows nodes being removed in stages. Initially, node A is selected. After its removal, nodes B and C become eligible. Finally, node D is processed.

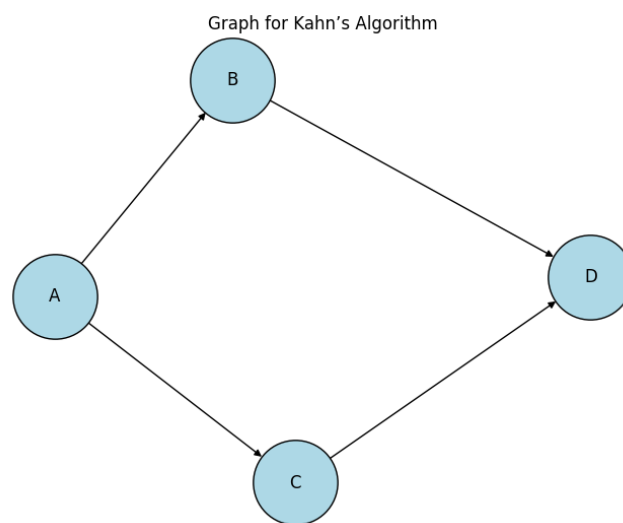


Figure 6.6.1 Step-by-step removal of nodes in Kahn's algorithm.

6.7 Implementation of Kahn's Algorithm

```
from collections import deque

def topological_sort_kahn(graph):
    # Step 1: compute in-degrees
    in_degree = {node: 0 for node in graph}

    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1

    # Step 2: queue nodes with in-degree 0
    queue = deque([node for node in graph if in_degree[node] == 0])

    order = []

    # Step 3: process the queue
    while queue:
        node = queue.popleft()
        order.append(node)

        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    return order

# Example DAG
graph = {
    "Get Bread": ["Add Butter"],
    "Add Butter": ["Sandwich Ready"],
    "Boil Water": ["Add Tea Leaves"],
    "Add Tea Leaves": ["Tea Ready"],
    "Sandwich Ready": [],
    "Tea Ready": []
}

# THIS LINE PRODUCES OUTPUT
print(topological_sort_kahn(graph))
```

Output:

```
['Get Bread', 'Boil Water', 'Add Butter', 'Add Tea Leaves',
'Sandwich Ready', 'Tea Ready']
```

6.8 Example Execution

```
graph = {  
    "A": ["B", "C"],  
    "B": ["D"],  
    "C": ["D"],  
    "D": []  
}  
  
print(topological_sort_kahn(graph))
```

Output

A
B
C
D

6.9 Method 2: DFS-Based Topological Sorting

6.9.1 Core Idea

In this method:

- Perform DFS
- Add nodes to a stack after exploring all neighbours
- Reverse the stack

6.9.2 Intuition

A node is added only after all tasks depending on it are processed.

6.9.3 Implementation

```
def topological_sort_dfs(graph):
    visited = set()
    stack = []

    def dfs(node):
        if node not in visited:
            visited.add(node)
            for neighbor in graph[node]:
                dfs(neighbor)
            stack.append(node)

    for node in graph:
        dfs(node)

    return stack[::-1]

# Example DAG for topological sorting
graph = {
    "Get Bread": ["Add Butter"],
    "Add Butter": ["Sandwich Ready"],
    "Boil Water": ["Add Tea Leaves"],
    "Add Tea Leaves": ["Tea Ready"],
    "Sandwich Ready": [],
    "Tea Ready": []
}

# Output
print(topological_sort_dfs(graph))
```

Output:

```
['Boil Water', 'Add Tea Leaves', 'Tea Ready', 'Get Bread', 'Add Butter', 'Sandwich Ready']
```

6.10 Dependency Graphs

A **dependency graph** is a directed graph where:

- Nodes represent tasks
- Edges represent dependencies

If A depends on B: $\rightarrow$ Edge: $B \rightarrow A$

Figure 6.10.1 shows how dependencies are modelled using directed edges.

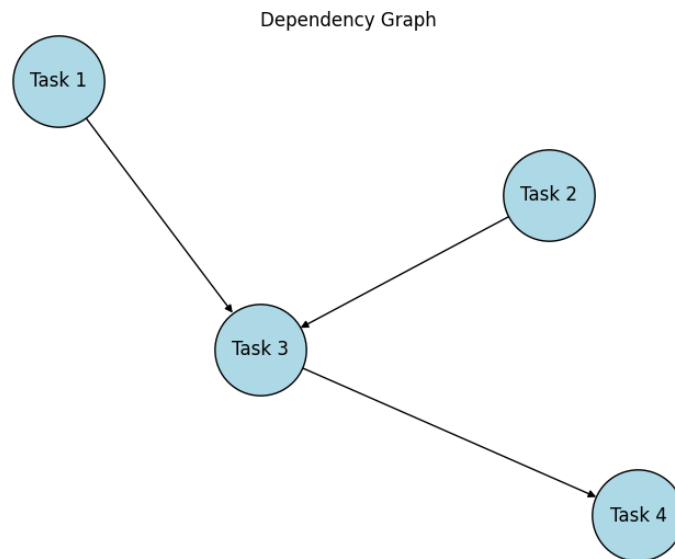


Figure 6.10.1: Dependency Graph

Tasks are connected such that prerequisite tasks point toward dependent tasks.

6.11 Key Insight

Topological sorting and dependency graphs together provide:

- A structured execution order
- A way to ensure correctness
- A foundation for workflow systems

6.12 Summary

In this chapter, we learned:

- Topological sorting produces a valid execution order
- It works only for acyclic graphs
- Kahn's algorithm uses in-degree
- DFS provides an alternative approach
- Dependency graphs model real-world systems

In the next chapter, we bring everything together: Directed Acyclic Graphs (DAGs) and State Flow.

Directed Acyclic Graphs (DAGs) and State Flow

In the previous chapters, we gradually built a complete understanding of graph-based systems. In Chapter 6, we studied **topological sorting**, which gives a valid execution order — but only when no cycles exist.

This naturally leads to a central concept:

What do we call a graph that has direction and **no cycles**?

Such a graph is known as a:

Directed Acyclic Graph (DAG)

7.1 What is a DAG?

A **Directed Acyclic Graph (DAG)** is a graph that satisfies two conditions:

1. All edges have direction (it is a directed graph)
2. It contains **no cycles** (it is acyclic)

In simple terms:

A DAG is a graph where you can never return to the same node by following directed edges.

7.2 Why DAGs Are Important

DAGs are extremely important because they guarantee:

1. No Infinite Loops

Since there are no cycles, execution always finishes.

2. Valid Execution Order

Topological sorting is always possible.

3. Clear Dependency Flow

Every task has a well-defined position.

7.3 Intuition Using Our Running Example

Let us revisit the cooking workflow:

Get Bread → Add Butter → Serve Sandwich

Boil Water → Add Tea Leaves → Serve Tea

We can combine both into a larger structure:

- Sandwich and tea preparation happen independently
- Both lead to a final goal: Serve Meal

This forms a **DAG** because:

- All dependencies are directional
- No cycles exist

7.4 Visual Representation of a DAG

Figure 7.4.1 illustrates a DAG where tasks flow toward a final outcome without forming cycles.

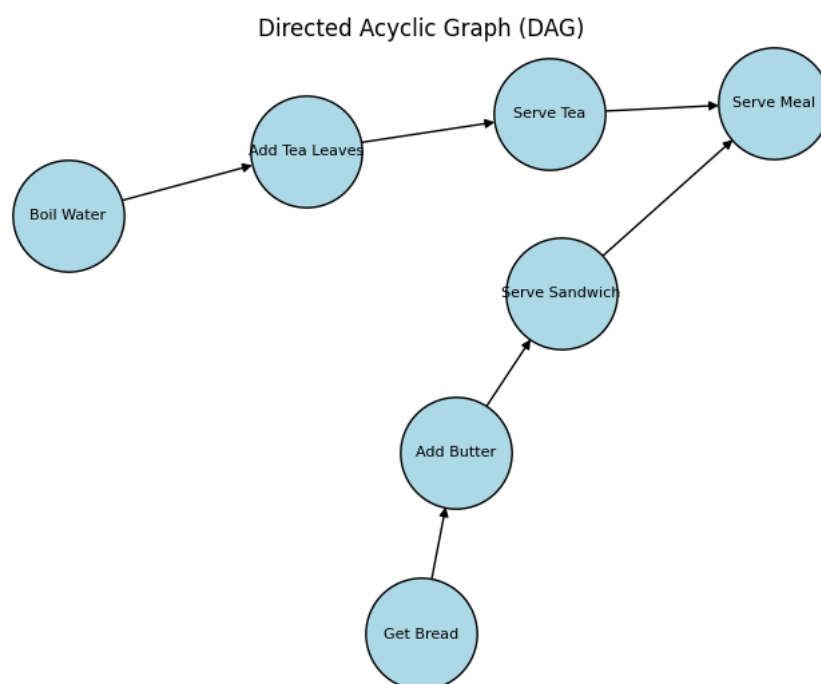


Figure 7.4.1: Directed Acyclic Graph (DAG)

The diagram shows multiple branches of tasks (sandwich and tea preparation) merging into a final node labelled “Serve Meal”. Arrows indicate direction, and no loops are present.

7.5 DAG vs General Graph

Feature	DAG	General Graph
Direction	Yes	Yes/No
Cycles	Not allowed	Allowed
Execution	Safe	May fail
Ordering	Always possible	Not always

7.6 DAG and Topological Ordering

From Chapter 6, we know:

Topological sorting is only possible if the graph has no cycles.

Therefore:

Every DAG has at least one valid topological ordering.

This means DAGs are ideal for:

- Task scheduling
- Workflow execution
- Program dependencies

7.7 Understanding State Flow in Graphs

So far, we have treated nodes as tasks. Now we extend this idea.

In many systems, nodes represent **states**, and edges represent **transitions between states**.

Example

Consider a simple process:

Start → Process Data → Analyze → Output

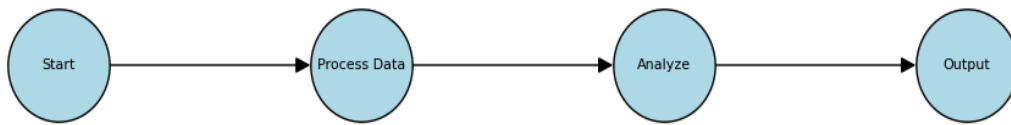


Figure 7.7.1: State Transition Graph Showing System States

Here:

- Each node represents a **state of the system**
- Each edge represents a **transition**

7.8 What is State Flow?

State flow refers to how a system moves from one state to another through a graph.

In a DAG:

- State flow is always **forward-moving**
- No backward loops exist
- Execution is predictable

Visualizing State Flow:

Figure 7.8.2 illustrates state flow in a DAG where the system transitions through multiple states, including branching and merging, before reaching a final output.

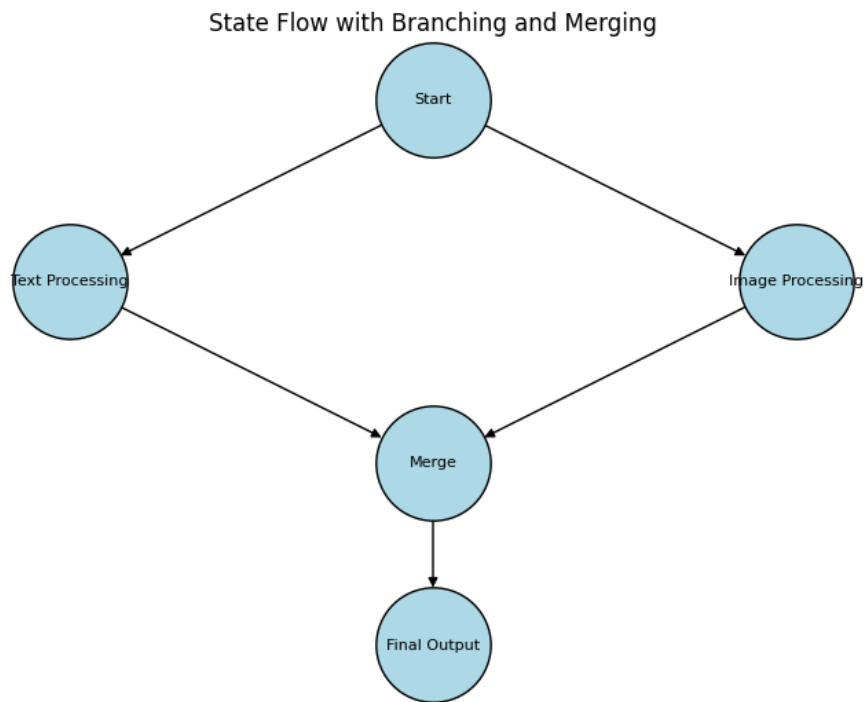


Figure 7.8.2: State Flow with Branching and Merging

The graph begins at a "Start" state. It then splits into two parallel paths:

- One path processes text data
- Another path processes image data

Both paths later merge into a single "Final Output" state.

This clearly shows:

- State transitions
- Parallel processing
- Convergence into a final state

7.9 Key Characteristics of State Flow in DAGs

1. **Unidirectional Movement**
Execution always moves forward.
2. **No Repetition**
Nodes are not revisited.
3. **Deterministic Behaviour**
The flow is predictable.

7.10 DAGs in Real-World Systems

DAGs are widely used in:

- Data processing pipelines
- Build systems (e.g., compiling code)
- Workflow automation
- AI pipelines

In all these systems:

- Tasks depend on previous tasks
- Execution must be correct and finite

7.11 Implementation Section: Verifying a DAG and Executing Flow

Step 1: Graph Representation

```
graph = {  
    "Start": ["Process"],  
    "Process": ["Analyze"],  
    "Analyze": ["Output"],  
    "Output": []  
}
```

Step 2: Check if Graph is a DAG

We use cycle detection (from Chapter 5):

```
def is_dag(graph):  
    visited = set()  
    path = set()  
  
    def dfs(node):  
        if node in path:  
            return False  
        if node in visited:  
            return True  
  
        visited.add(node)  
        path.add(node)
```

```
#.....(continued)

    for neighbor in graph[node]:
        if not dfs(neighbor):
            return False

    path.remove(node)
    return True

for node in graph:
    if not dfs(node):
        return False

return True
```

Output

True

Step 3: Execute Using Topological Order

```
def topo_sort(graph):
    visited = set()
    stack = []

    def dfs(node):
        if node not in visited:
            visited.add(node)

            for neighbor in graph[node]:
                dfs(neighbor)

            stack.append(node)

    for node in graph:
        dfs(node)

    return stack[::-1]

print(topo_sort(graph))
```

Output

Start
Process
Analyze
Output

7.12 Key Insight

A DAG is not just a graph structure. It represents:

A safe, executable flow of states or tasks

It combines:

- Dependency management
- Execution order
- Flow control

7.13 Summary

In this chapter, we learned:

- DAGs are directed graphs with no cycles
- They guarantee safe execution
- Topological sorting always works on DAGs
- State flow represents movement through nodes
- DAGs are widely used in real-world systems

LangGraph and Agent Planning: Applying Graph Foundations

Across this textbook, we built our understanding step by step:

- Graphs represent relationships and dependencies
- Trees simplify hierarchical structures
- Traversals define execution flow
- Cycles break systems and must be avoided
- Cycle detection ensures safety
- Topological sorting provides execution order
- DAGs guarantee correct and finite execution
- State flow models how systems evolve

Now we bring everything together into a real-world system:

Graph-based AI execution using DAGs

This is exactly what modern frameworks like **LangGraph** and agent systems rely on.

8.1 What is LangGraph? (Conceptual View)

LangGraph is a framework for building systems where:

- Tasks are represented as **nodes**
- Execution flow is defined using **edges**
- The entire system behaves like a **graph**

At its core:

LangGraph uses **graph structures (especially DAGs)** to control how an AI system executes.

8.2 Mapping Graph Concepts to AI Systems

Let us map what we learned to AI systems:

Graph Concept	Meaning in AI Systems
Node	LLM call / tool / function
Edge	Dependency or transition
Graph	Workflow
DAG	Safe execution plan
Traversal	Execution process
State	Intermediate output

8.3 Example: AI Chatbot Pipeline

Let us build a simple chatbot workflow:

1. Take user input
2. Process input
3. Decide action
4. Generate response
5. Output result

Graph Representation

User Input → Process → Decision → Response → Output

Visual Representation

Figure 8.3.1 shows how a chatbot pipeline can be modelled as a graph of sequential states.

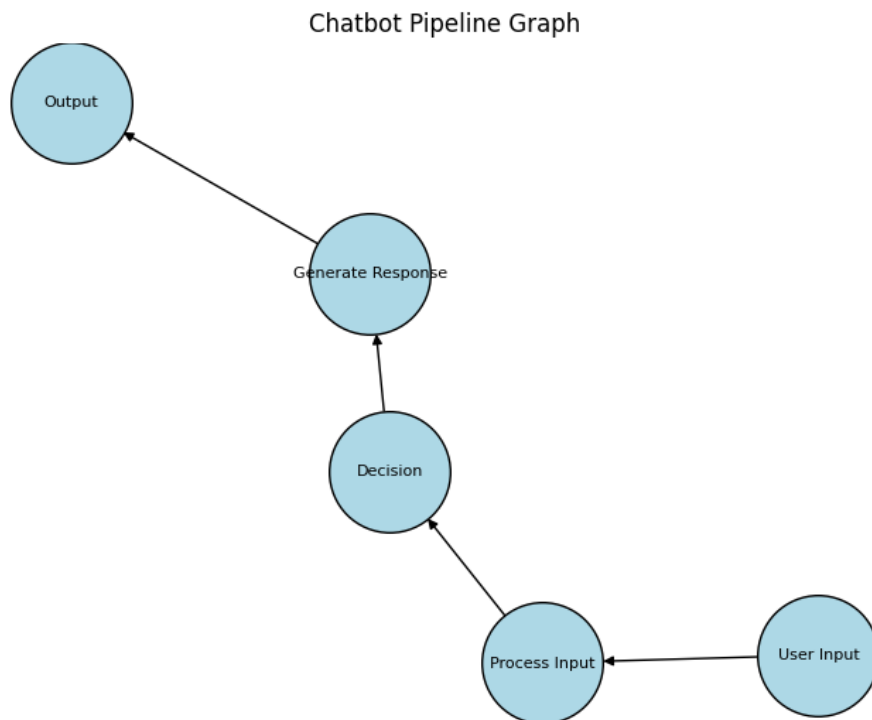


Figure 8.3.1: Chatbot Pipeline as a Graph

The diagram shows a linear flow of nodes starting from “User Input” and ending at “Output”. Each node represents a stage in processing.

8.4 Introducing Branching in Agent Systems

Real AI systems are not always linear. They often include decisions.

For example:

- If question is simple → answer directly
- If complex → call external tool

Graph with Decision Branching

Figure 8.2 shows how agent systems branch based on decisions.

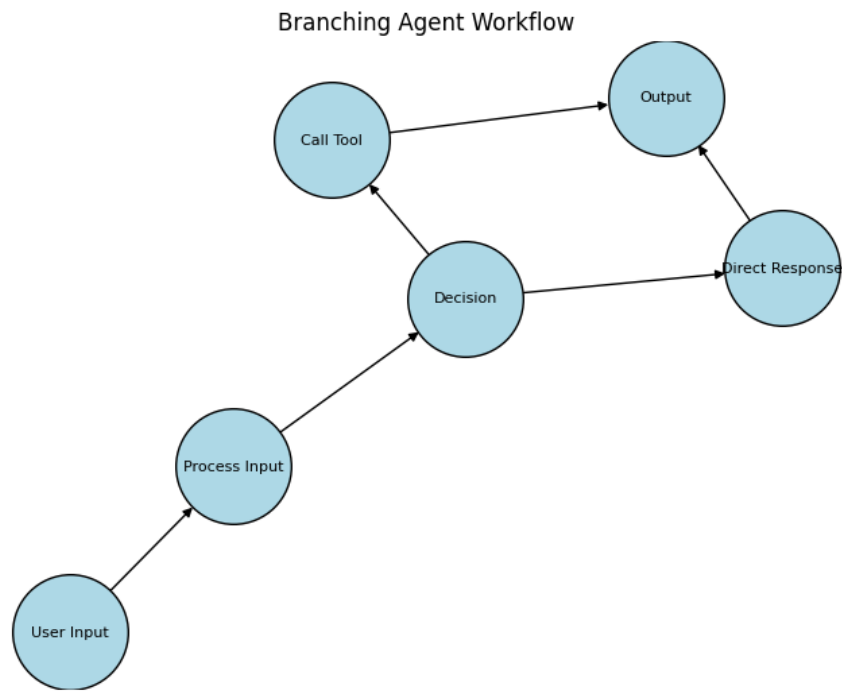


Figure 8.4.1: Branching in Agent Workflow

The graph branches at the “Decision” node into two paths: one leading to a direct response and another leading to a tool call. Both paths eventually lead to output.

8.5 Why DAGs Are Critical in LangGraph

From Chapter 7, we know:

DAGs guarantee safe and finite execution.

In AI systems, this is essential because:

1. Prevent Infinite Loops

Agents must not repeat endlessly.

2. Ensure Correct Order

Dependencies must be respected.

3. Enable Debugging

Clear flow makes systems understandable.

8.6 State Flow in Agent Systems

In LangGraph, the system moves through **states**:

- Input state
- Processed state
- Decision state
- Output state

Each node:

- Takes a state as input
- Produces a new state

Key Idea

Nodes transform state, edges move state forward.

Visualizing State Flow in Agent System

Figure 8.6.1 illustrates how state flows through an AI system modelled as a DAG.

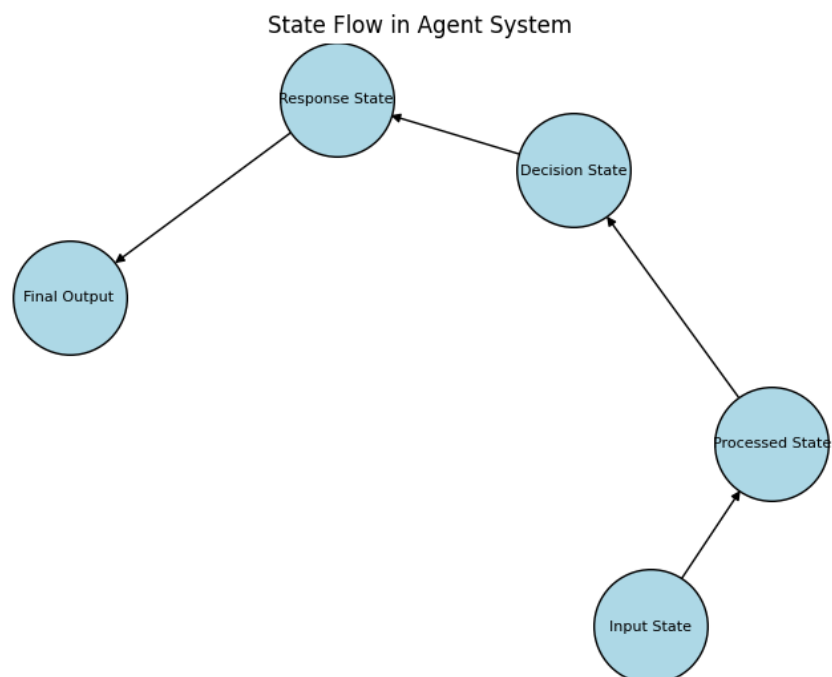


Figure 8.6.1: State Flow in Agent Pipeline

The graph shows how input state evolves through processing, decision-making, and response generation, eventually reaching a final output state.

8.7 Implementation Section: Simple Agent Workflow

Step 1: Define Workflow as Graph

```
graph = {  
    "Input": ["Process"],  
    "Process": ["Decision"],  
    "Decision": ["Respond", "Use Tool"],  
    "Respond": ["Output"],  
    "Use Tool": ["Output"],  
    "Output": []  
}
```

Step 2: Execute Using Topological Sort

```
def topo_sort(graph):  
    visited = set()  
    stack = []  
  
    def dfs(node):  
        if node not in visited:  
            visited.add(node)  
  
            for neighbor in graph[node]:  
                dfs(neighbor)  
  
            stack.append(node)  
  
    for node in graph:  
        dfs(node)  
  
    return stack[::-1]  
  
print(topo_sort(graph))
```

Output:

['Input', 'Process', 'Decision', 'Use Tool', 'Respond', 'Output']

Step 3: Simulated Execution

```
def execute(graph, order):  
    for node in order:  
        print(f"Executing: {node}")  
  
order = topo_sort(graph)  
execute(graph, order)
```

Output

Executing: Input

Executing: Process

Executing: Decision

Executing: Use Tool

Executing: Respond

Executing: Output

8.8 Mini Project

Convert a Real-World Workflow into a DAG

Task

Choose a real-world process such as:

- Online shopping
- Exam preparation
- Food delivery system

Steps

1. Identify all tasks
2. Define dependencies
3. Draw the graph
4. Ensure no cycles exist
5. Find topological order

Example: Exam Preparation

- Study → Practice → Revise → Exam

Challenge

Extend it:

- Add branching (weak topics vs strong topics)
- Add merging (final revision)

8.9 Final Summary

You have now learned:

- Graph fundamentals
- Traversal methods
- Cycles and their dangers
- Cycle detection techniques
- Topological sorting
- Dependency modeling
- DAGs and state flow
- Application in AI systems

You are now ready to:

- Learn LangGraph practically
- Build agent-based systems
- Design workflows using DAGs
- Understand execution flow deeply